LATTICEFORGE SYSTEM ARCHITECTURE

Exact Capability Routing via Galois Closure & Multi-Objective Pareto Optimal Agent Execution

1 USER & TASK INGESTION LAYER

Multi-Modal Inputs • Execution Preferences • Priority Budgets

User Task Prompt — INPUT

- Natural language instruction
- Composite requirements (PDF, SQL, Code)
- Unstructured operational queries

Priority Weights (w) — CONSTRAINTS

- **Quality Weight (w_q):** Maximize output accuracy
- **Latency Weight (w_λ):** Minimize response time
- **Cost Weight (w_γ):** Minimize token consumption

User Tier & SLA Contract — POLICY

- **Enterprise:** Low latency & high fidelity priority
- **Batch / Free:** Cost optimization focus
- **Timeout / Budget:** Strict cost ceiling cap

Task String + User Priority Vector $w = (w_q, w_\lambda, w_\gamma)$ ↓

2 LATTICEFORGE CORE MATHEMATICAL SUBSYSTEM

Formal Concept Analysis (FCA) & Pareto Frontier Optimization

2.1 Capability Extractor — FCA PARSER

- Few-shot semantic attribute classification
- Maps task tokens to atomic capabilities:
- `M = {sql_access, code_exec, web_search, pdf_parse, ...}`

`Target Set: B_req ⊆ M`

2.2 Discrete Galois Engine — PROVABLE

- **Formal Context:** K = (G, M, I)
- **Derivation:** B' = {t ∈ G | ∀a ∈ B, tIa}
- **Galois Closure:** $B''_{req} = (B'_{req})'$
- **Coverage Theorem:** CoverageError = 0

`Optimal Concept: (A*, B*)`

2.3 Pareto Frontier Engine — OPTIMIZER

- Evaluates non-dominated configurations:
- `P* = {c ∈ C | ∄ c', c' ≻ c}`
- **Scalarization:** Min weighted L2 distance to Ideal Point Z* = (1, 0, 0)

`Optimal Config: c* ∈ P*`

Covering Concept Tools Extent A* + Selected Execution Config c* ↓

3 DYNAMIC DISPATCH & MIDDLEWARE GATEWAY

Model Routing • Prompt Context Serialization • Safety Sandbox

Minimal Schema Injector — CONTEXT

- Serializes only verified tools in A*
- Eliminates 'lost-in-middle' distractors
- Zero quadratic prompt token bloat

Multi-Model Router — INFERENCE

- **Groq Hardware:** Llama-3.3-70B (High speed)
- **Frontier:** GPT-4o / Claude 3.5 Sonnet
- Dynamically bound by Pareto selection

Execution Guardrails — SECURITY

- Strict API permission boundaries
- Sub-process isolation & rate limits
- Deterministic output validation

Inference Query with Minimal Tool Schemas ↓

4 AGENT EXECUTION & HETEROGENEOUS TOOL SANDBOX

Deterministic Execution Loop • Microservices • Domain APIs

SQL Analytics — API

- Capabilities: `{sql_access, data_analysis}`
- Enterprise database connector
- Aggregations & query plans

Python Interpreter — SANDBOX

- Capabilities: `{code_exec, data_analysis}`
- Secure isolated sandbox
- Data modeling, plotting, math

Web Scraper — WEB

- Capabilities: `{web_search}`
- Live web crawling & parsing
- Real-time financial/news data

PDF / Doc Parser — DOC

- Capabilities: `{pdf_parse, data_analysis}`
- Extracts schemas & tables
- Unstructured doc processing

5 AUTONOMOUS SELF-REFACTORING

Continuous Telemetry & Formal Maintenance

Live Telemetry Engine

Continuous SQLite logging of per-query latency (ms), token cost ($), and execution quality (0-1).

Redundancy Verifier

Mathematically proves if tool T can be safely retired without capability loss across all tasks.

`∀t: tIa ⇒ ∃ G' ⊆ G - {T}`

Capability Gap Detector

Identifies frequently requested attribute subsets where Extent B' = ∅, triggering new tool alerts.

`B' = ∅ ⇒ Flag Gap Alert`

Closed-Loop Adaptation

- Nightly recomputation of Pareto frontiers from empirical latency/cost distributions.
- Incremental Next-Closure update of Formal Context K = (G, M, I).